

Unveiling LLM Hallucination in Recommending Third-party Packages

Abstract

In this paper, ...

1 Introduction

1. LLM widely used for developers
2. LLM hallucination
3. Hallucinated packages can cause supply chain attack.
4. How hallucinated package is not yet fully explored
5. The purpose of this study is to.
6. Specifically, we design xxx.
7. The results reveal that
8. Our contribution are as follows.

2 Background

2.1 Hallucination

2.2 Package Squatting

3 Empirical Setup

3.1 Setup

3.1.1 Constructing the Functional Question Dataset. To investigate the hallucination phenomenon of LLMs in recommending Python packages, we first needed to construct a representative dataset of functional questions. Our goal was to identify questions from Stack Overflow that are related to popular Python development scenarios and are likely to trigger package recommendation behavior.

First, we followed the co-occurrence-based tag selection method proposed by Huang et al.[3] and used the public Stack Overflow dataset provided by Google BigQuery[2] to extract all tags that co-occurred with the python tag more than 1,000 times. This yielded a total of 212 tags. Building upon this, we were inspired by the task topic classification strategy used by Du et al.[1] in designing code generation benchmarks, and we referred to the categorization of popular Python development topics proposed by Tahmoore et al.[4]. We manually analyzed the 212 high-frequency co-occurring tags to determine whether each tag belonged to one of the 12 mainstream development domains, whether it was likely to involve the use or recommendation of Python packages, and whether it might give rise to “how to implement” type functional questions. Based on this screening, we retained 114 tags. The detailed categorization is shown in Table ??.

Based on this, we extracted all questions from the public dataset that contained both the python tag and at least one of the 114 selected tags, resulting in a total of 287,944 questions. We randomly sampled 200 of these questions for manual analysis and found that typical questions related to package recommendation often included functional expressions such as “how to” or “how can I”. Therefore, we used these keywords

to filter all potentially functional questions from the dataset, yielding 49,967 candidate questions.

To further filter out functional questions from the 49,967 candidates, we conducted a detailed analysis on a random sample of 200 questions to summarize the typical characteristics of functional questions. Based on these characteristics, we designed a targeted prompt and used the large language model DeepSeek R1 to determine whether each question qualified as a functional question. As a result, we obtained a refined set of 17,696 functional questions. Details of the prompt design can be found in Appendix A.

To further filter the selected functional questions, we removed those that did not have an accepted answer, ensuring that each retained question had a verified solution. We then extracted the accepted answer for each question from the public dataset and combined it with the corresponding question to form question-answer pairs.

To identify whether each accepted answer involved specific third-party Python libraries, we designed a prompt (see Appendix B for details). We then used the large language model DeepSeek R1 to analyze each accepted answer and extract any mentioned third-party library names (i.e., packages installable via pip from PyPI). If the model returned null (indicating that no third-party libraries were involved), then the corresponding question-answer pair was filtered out in the subsequent processing. As a result, we obtained 4,382 questions to form our final functional question dataset.

Since the above question-answer pairs were collected from the public Stack Overflow community, they might have already been seen by large language models during training. As a result, the hallucination problem of virtual packages might be more severe than what our initial experiments suggest. To obtain more realistic experimental results, we employed a diffusion-based text generation approach to rewrite the question titles. Diffusion models have achieved remarkable success in handling continuous signals such as vision and audio, and due to their — characteristics, they also show great potential in sequence-to-sequence tasks. Some studies have demonstrated that diffusion models can achieve outstanding results in terms of both generation quality and output diversity.

In summary, we constructed a functional question-answer dataset based on the public Stack Overflow data, covering a wide range of typical Python development scenarios.

3.1.2 Querying LLMs for Package Suggestions. To evaluate the frequency at which LLMs hallucinate packages when recommending Python libraries, we designed a unified and comprehensive experimental pipeline. We conducted experiments on several mainstream models, including ChatGPT-4, DeepSeek-R1, Qwen3, and Claude-Opus-4. For each model, we used the titles of questions from our previously constructed

functional question dataset as input prompts, asking the models to recommend one or more third-party Python libraries that could be used to solve the given problem.

We first designed a unified prompt for all models, with the full content detailed in Appendix C. During the querying process, we observed that some models returned package names containing characters that are not allowed on PyPI (e.g., parentheses, brackets, equal signs, or spaces). To handle this, we implemented an automated script to detect such invalid outputs and re-issued the queries until the returned package names no longer contained illegal characters.

In addition, for some package names generated by the models that contained dot characters (e.g., `scipy.sparse`), we found that such submodules could not be directly installed as top level packages. To address this, we designed a more restrictive prompt (see Appendix D), which explicitly instructed the model to return only top level package names, for example, replacing `scipy.sparse` with `scipy`. Whenever a returned result contained a dot, we re-issued the query using this stricter prompt. However, we observed that even after multiple rounds of clarification, a few models still occasionally returned package names with dots (such as `RPI.GPIO`). In such cases, we chose to retain these results and handle them consistently in later stages of our process.

After completing all queries, we uniformly processed the returned package names. First, we removed all records with the return value `no_package`, retaining only those questions for which the model explicitly recommended package name. Then, we queried each recommended package individually using PyPI's official JSON API https://pypi.org/pypi/{pkg_name}/json to check whether the package truly exists. If the request returned a 200 status code, the package was considered real. During this process, we noticed that package names containing a dot, which had previously been retained, also received 200 responses from the API. This indicated that these dotted package names were indeed present on PyPI, and we thus treated them as real packages as well.

Finally, we conducted further screening on the package names that failed the PyPI check and were initially identified as hallucinated packages. Although our prompt explicitly instructed the models to recommend third-party libraries, a small number of Python standard libraries (e.g., `os`, `sys`) were still suggested by some models. Since these standard libraries cannot be found on PyPI, they were mistakenly classified as hallucinated packages. To eliminate such cases, we used the `importlib.util.find_spec()` interface to determine whether a given package was part of the standard library. After filtering out standard libraries through this additional step, we ultimately obtained a result set that contained only hallucinated packages.

3.1.3 Registering Hallucinated Packages to PyPI. To further investigate whether hallucinated packages generated by LLMs pose a risk of being downloaded by real developers, we selected 91 hallucinated package names recommended by ChatGPT-4 and attempted to actually register and upload them to the PyPI platform. For each package, we prepared

standardized versions of the `setup.py`, `__init__.py`, and `README.md` files, explicitly stating that the package was created solely for academic research purposes, contained no functional code, and performed no malicious operations. The `README` clearly described the research context, researchers involved, contact information, and research objectives, ensuring full compliance with academic ethics. All packages were uploaded manually by us.

We began uploading the packages on May 18, 2025, and completed the process by May 22, 2025. A total of nine PyPI accounts were used to upload the packages in batches. Among the 91 packages, 55 were successfully registered, while the remaining 36 failed due to various reasons. These failures included package names being protected by PyPI, names being too similar to existing projects, or accounts being restricted by the platform due to suspicious upload activities.

On June 10, 2025, we received an email from PyPI administrator requesting the immediate removal of all experiment-related packages. In response, we took down all packages during the period from June 12 to June 13.

After uploading the packages to PyPI, we tracked their download activity to observe the actual impact of these hallucinated packages. We used two third-party analytics platforms, `pepy.tech` and `pypistats.org`, to collect the download statistics for each package. These platforms provide both daily download counts and cumulative downloads over different time periods. The corresponding results and analysis are presented in the **Result** section.

3.2 Research Questions

- **Hallucination Degree (RQ1):** To what extent do LLMs generate hallucinated package names when recommending third-party packages?
- **Hallucinated Package Characteristics (RQ2):** What are the characteristics of the hallucinated package?
- **Distribution Severity (RQ3):** How do hallucinated packages survive and distribute on the package registry?
- **Unseen Query Evaluation (RQ4):** How do LLMs perform regarding queries that are unseen in the dataset?

To study RQ1, we used our constructed dataset of functional questions to uniformly query several mainstream LLMs, including ChatGPT-4, DeepSeek-R1, Qwen3, Claude-Opus-4, asking them to recommend third-party Python libraries that could help solve each question. We then verified the recommended packages using the PyPI API to determine whether they actually exist, and calculated the hallucination frequency for each model accordingly.

To study RQ2, we applied a diffusion-model-based rewriting method to modify the titles of the functional questions and then used the same querying procedure to prompt each LLM with these rewritten questions. This allows us to assess whether the models are more likely to generate hallucinated packages when facing "unseen" questions.

To study RQ3, we registered the hallucinated packages recommended by ChatGPT-4 on the PyPI platform and monitored their download activity over a specific period.

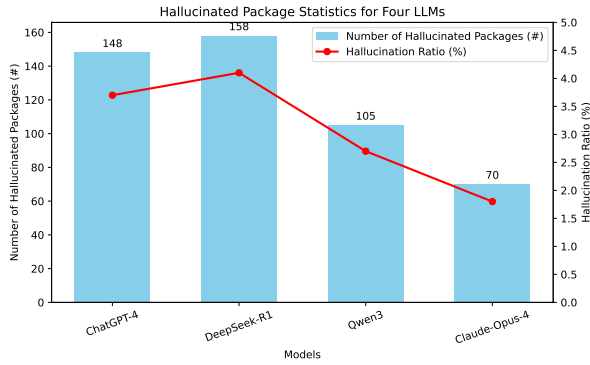


Figure 1: The number and ratio of hallucinated packages generated by four LLMs.

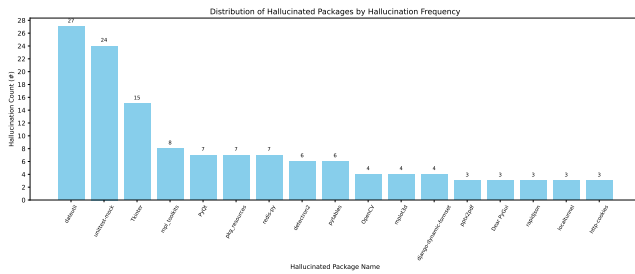


Figure 2: Distribution of hallucinated packages by hallucination frequency.

Using data from third-party analytics platforms, we tracked how many times these packages were downloaded by real users or automated tools, in order to assess the potential risk of them being exploited by attackers to deceive real users into downloading and using them.

4 Results

4.1 RQ1

1. Fig: x-axis is models, y-axis are hallucinated package absolute number and relative hallucination percentage.

2. Fig: x-axis is hallucinated packages ranked by their hallucination times among models, y-axis is the hallucination number.

4.2 RQ2

1. Package name characteristics.

1.1 Syntactic analysis 1.2 Package domain analysis

2. Analysis on questions that cause hallucinated package name.

4.3 RQ3

x-axis is time, y-axis is total download times, and average download number.

TODO fig. download numbers with regard to package names

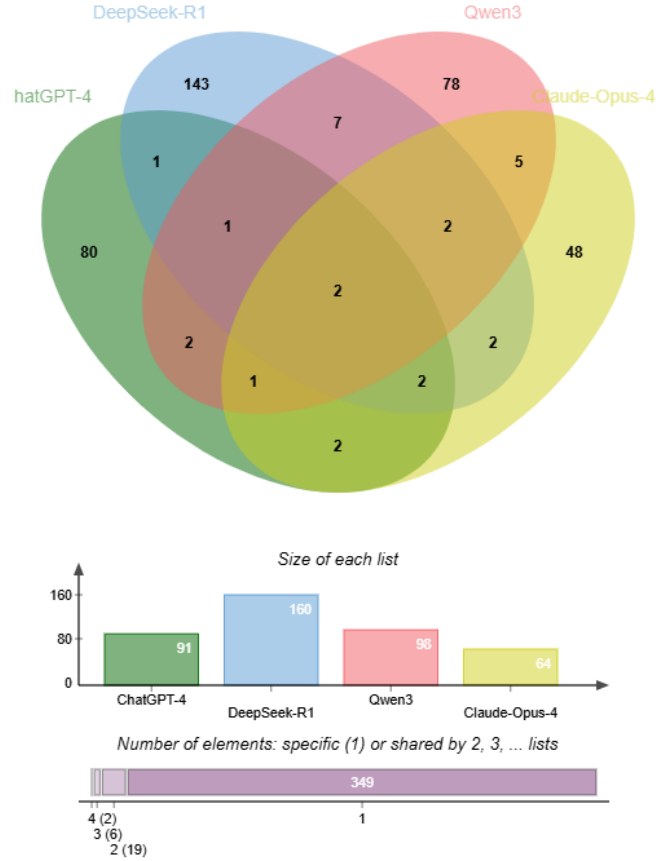


Figure 3: Venn diagram of hallucinated package overlaps among LLMs.

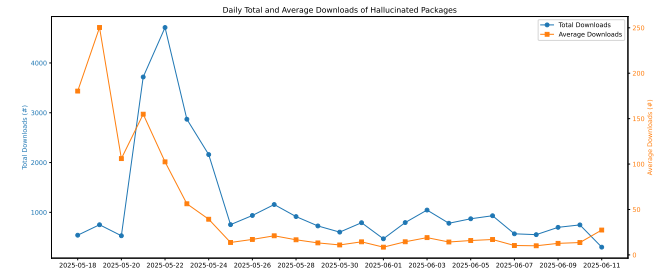


Figure 4: Daily total and average downloads of hallucinated packages.

4.4 RQ4

Put Table, illustrate characteristics.

5 Mitigation

5.1 Mitigation Methodology

5.1.1 Enhanced Prompting.

5.1.2 Machine-learning-based Detection.

Table 1: Feature Categories of Hallucinated PyPI Package Names

Feature Category	Example Package Names	Count
PyPI Submodules	mpl_toolkits, mplot3d, gridsfs, pkg_resources, sklearn-base, django-forms	6
Function/Usage Names	odeint, django-contenttypes	2
Misspelled Packages	urllib3, shutilwich, maze-solvers	3
Fabricated Based on Real PyPI Packages	django-classy-views, flask-autocomplete	20+
Import Name Mismatch	MySQLdb, yaml, dateutil	3
Old Package Names	pytest-capturelog	1
Standard Library Names	email-parser, osrandom, redirect-stdout, BytesIO	4
Legacy Stdlib Names	Tkinter	1
Colloquial or Informal Names	redis-py, pyqt, opencv	3
Derived from Technical Terms	translate-api, pycapsule-api, manifestin	3
Platform / C Library / Tools	zeromq, OpenSSL, package_control, h2oai, tastypie, sslsplit	6

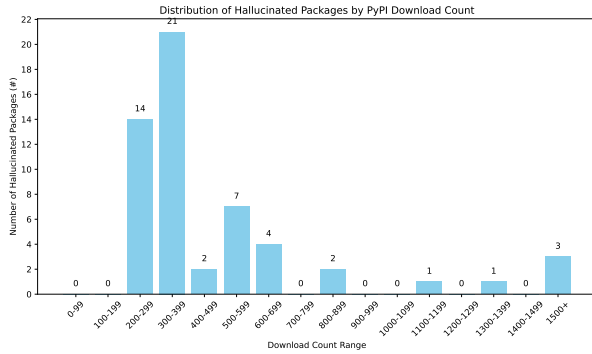


Figure 5: Distribution of hallucinated packages by total PyPI downloads.

5.1.3 Context-Augmented Validation. Typo squatted package?

5.2 Evaluation Setup

- **Mitigation Effectiveness (RQ5):** How do mitigation techniques address the hallucinated package problem?

5.3 RQ5

5.4 Reservation of Squating/Hallucination Packages

6 Related Work

7 Conclusion

References

- [1] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE/ACM, 1–13. <https://doi.org/10.1145/3597503.3639219>
- [2] Google Cloud Platform. [n.d.]. Stack Overflow: BigQuery Public Dataset. <https://console.cloud.google.com/bigquery?project=bigquery-public-data>. Accessed July 2025.
- [3] Kaifeng Huang, Bihuan Chen, Susheng Wu, Junming Cao, Lei Ma, and Xin Peng. 2023. Demystifying Dependency Bugs in Deep Learning Stack. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 450–462. <https://doi.org/10.1145/3611643.3616325>
- [4] Hamed Tahmooreesi, Abbas Heydarnoori, and Alireza Aghamohammadi. 2020. An Analysis of Python’s Topics, Trends, and Technologies Through Mining Stack Overflow Discussions. *arXiv preprint arXiv:2004.06280* (2020). <https://arxiv.org/abs/2004.06280>

Table 2: AUROC (%) of different methods under varying member-to-non-member ratios.

Ratio	Method	Pythia 2.8B	GPT-Neo 2.7B	StableLM-Alpha 3B	GPT-J 6B
1:1	LOSS	38.4	43.7	33.3	43.3
	ZLIB	33.2	35.5	32.0	35.7
	MIN-K	38.8	41.8	34.2	41.1
	DC-PDD	50.1	41.2	43.7	41.3
	TOOL	61.3	59.7	61.3	59.7
1:5	LOSS	40.3	45.3	33.9	44.8
	ZLIB	34.0	36.2	32.5	36.3
	MIN-K	40.2	43.3	34.9	42.4
	DC-PDD	51.8	44.0	44.6	43.2
	TOOL	61.2	59.4	61.1	61.5
5:1	LOSS	40.0	43.8	34.9	43.6
	ZLIB	33.1	35.0	31.9	35.2
	MIN-K	39.9	42.2	35.1	41.6
	DC-PDD	49.4	40.9	44.1	39.6
	TOOL	62.0	60.7	62.0	63.1